Лабораторная работа №2: Реализация композиции против наследования: проектирование системы тарификации и платежей

Курс: Объектно-ориентированное программирование 1 (CS2203)
Университет: КазНУ им. аль-Фараби
Преподаватель: Санжар Аубакиров, PhD
Неделя: 2 из 10
Формат: Лабораторная работа, 3 академических часа (135 минут)
Среда выполнения: Python 3.12+, pytest, Git, VS Code / PyCharm

---

1. Цель и контекст работы

Цель работы: На практическом примере разобрать антипаттерн «комбинаторный взрыв классов при наследовании» и провести рефакторинг подсистемы расчета стоимости проезда (Tariff & Pricing Engine) на компонентную модель с использованием принципа **«Composition over Inheritance»**.
В системе Avtobys стоимость поездки зависит от нескольких независимых факторов: 1. Базовый тариф маршрута (городской, экспресс, пригородный). 2. Категория пассажира (обычный, студент 50%, пенсионер 100%, школьник 50%). 3. Временной фактор (час пик, ночное время, выходной день). 4. Способ оплаты (QR-код, транспортная карта, виртуальная банковская карта).
Попытка закодировать это в иерархию классов привела бы к созданию десятков классов вида SuburbanStudentPeakHourQrFare. Ваша задача — спроектировать модульную систему, где калькулятор собирается из легковесных заменяемых компонентов.

---

2. Структура репозитория и требования к сдаче

Работа выполняется в папке lab02_composition_pricing/ вашего репозитория:

```
lab02_composition_pricing/
├── pricing/
│   ├── __init__.py
│   ├── policies.py       # Интерфейсы и реализации политик (скидки, зоны, время)
│   ├── calculator.py     # Контекстный класс FareCalculator (композиция)
│   └── payment_gate.py   # Модуль шлюзов списания (ассоциация с TransitCard)
├── tests/
│   ├── __init__.py
│   └── test_pricing.py   # Тестовый сьют с параметризацией pytest
└── README.md             # Аналитический отчет: диаграмма до рефакторинга vs после
```

Критерии оценки (20 баллов):

Блок	Задание	Баллы
Блок 1	Анализ легаси-кода на наследовании и выявление слабых мест	4
Блок 2	Проектирование компонентной архитектуры политик через Protocols	8
Блок 3	Реализация параметризованных тестов pytest с проверкой граничных условий	5
Блок 4	Защита работы: объяснение жизненного цикла компонентов и trade-offs	3

---

3. Практические задания

Блок 1. Анализ антипаттерна наследования (4 балла)

В отчете README.md кратко опишите, почему представленная ниже иерархия нарушает принципы проектирования и к каким проблемам приведет добавление нового типа льготы («Ветеран»):

```
# АНТИПАТТЕРН: иерархия наследования
class BaseFare: ...
class StudentFare(BaseFare): ...
class SuburbanFare(BaseFare): ...
class StudentSuburbanFare(StudentFare, SuburbanFare): ... # Ромбовидное наследование!
```

Блок 2. Проектирование компонентного движка тарифов (8 баллов)

В файле pricing/policies.py создайте независимые протоколы и их реализации:

1. Политики скидок (DiscountPolicy):

- StandardDiscountPolicy: скидка 0% (полная оплата).
- StudentDiscountPolicy: скидка 50% от расчетной стоимости.
- SeniorDiscountPolicy: скидка 100% (бесплатный социальный проезд).
- ChildDiscountPolicy: скидка 50%.

2. Зональные коэффициенты (ZonePolicy):

- UrbanZonePolicy: коэффициент 1.00.
- SuburbanZonePolicy: коэффициент 1.30 (пригородное сообщение).
- ExpressZonePolicy: коэффициент 1.50 (экспресс-маршрут с минимумом остановок).

3. Временные модификаторы (TimePolicy):

- StandardTimePolicy: коэффициент 1.00.
- NightTimePolicy: коэффициент 1.20 (ночные дежурные маршруты с 23:00 до 06:00).

- PeakHourPolicy: динамический коэффициент загруженности.

4. Калькулятор тарифа FareCalculator (pricing/calculator.py):
Класс собирает компоненты через конструктор (Dependency Injection via Constructor) и рассчитывает стоимость:

$$\text{Итоговая сумма} = (\text{BaseRate} \times \text{ZoneMultiplier} \times \text{TimeMultiplier}) - \text{Discount}$$

```
class FareCalculator:
    def __init__(
        self,
        base_rate: Decimal,
        discount_policy: DiscountPolicy,
        zone_policy: ZoneMultiplierPolicy,
        time_policy: TimeMultiplierPolicy
    ):
        ...
```

Блок 3. Автоматическое тестирование (pytest) (5 баллов)

В файле tests/test_pricing.py напишите тесты, используя механизм @pytest.mark.parametrize:

```
import pytest
from decimal import Decimal

@pytest.mark.parametrize(
    "base, discount_cls, zone_cls, time_cls, expected",
    [
        (Decimal("100.00"), StandardDiscountPolicy, UrbanZonePolicy, StandardTimePolicy, Decimal("100.00")),
        (Decimal("100.00"), StudentDiscountPolicy, UrbanZonePolicy, StandardTimePolicy, Decimal("50.00")),
        (Decimal("100.00"), StudentDiscountPolicy, SuburbanZonePolicy, StandardTimePolicy, Decimal("65.00")),
        (Decimal("100.00"), SeniorDiscountPolicy, ExpressZonePolicy, NightTimePolicy, Decimal("0.00")),
        (Decimal("120.00"), StandardDiscountPolicy, SuburbanZonePolicy, NightTimePolicy, Decimal("187.20")),
    ]
)
def test_fare_calculation_combinations(base, discount_cls, zone_cls, time_cls, expected):
    ...
```

Все денежные вычисления должны быть строго типизированы через Decimal с округлением банковским методом или ROUND_HALF_UP до 2 знаков.

Блок 4. Вопросы для устной защиты (3 балла)

1. Чем отличается агрегация от композиции с точки зрения управления памятью и жизненным циклом объектов?
2. Почему множественное наследование в Python (поиск методов по MRO) может приводить к трудноуловимым багам в больших проектах?
3. Каким образом переход от наследования к композиции упростил тестирование в Блоке 3?

4. Как внедрение новой политики (например, скидка по промокоду выходного дня) затронет существующий класс FareCalculator?

---

4. Чек-лист проверки работы

- ☐ Отсутствуют монолитные иерархии наследования для комбинаций тарифов.
- ☐ Все вычисления проводятся с использованием Decimal, числа с плавающей точкой (float) исключены.
- ☐ Тестовое покрытие охватывает 100% заявленных комбинаций тарифов.
- ☐ Все классы снабжены корректными type hints и docstrings.